

UNIT -1

Overview of C++ language

1. C++ can be considered as an incremental version of c language which consists all programming
2. language constructs with newly added features of object oriented programming.
3. c++ is structure(procedure) oriented and object oriented programming language.
4. The file extension of C++ program is “.CPP”
5. Function overloading and operator overloading are possible.
6. Variables can be declared in inline i.e when required
7. In c++ more emphasis is give on data rather than procedures
8. Polymorphism, encapsulation and inheritance are possible.
9. Data abstraction property is supported by c++.
10. Data access is limited. It can be accessed by providing various visibility modes both for data
11. and member functions. there by providing data security by data hiding
12. Dymanic binding is supported by C++
13. .It supports all features of c language
14. It can be called as an incremental version of c language.

Programming Paradigms:-

Let us go on a whirlwind tour of 4 different programming paradigms –

- Procedural Programming Paradigms
- Object-Oriented Programming Paradigms
- Functional Programming Paradigms
- Logical Programming Paradigms

1. Procedural Programming Paradigm

Program is divided into procedures/functions.

Focuses on step-by-step execution.

Uses functions, loops, and conditions.

Example languages: C, Pascal

Example use: Simple programs, system-level programming

2. Logical Programming Paradigm

Program is written using facts and rules.
Execution is based on logic and inference.
Focuses on what to do, not how to do.
Example language: Prolog

Example use: AI, expert systems

3. Functional Programming Paradigm

Program is built using pure functions.
Avoids changing data and state.
Uses recursion instead of loops.
Example languages: Lisp, Haskell

Example use: Mathematical computations, data processing

4. Object-Oriented Programming (OOP) Paradigm

Program is organized using objects and classes.
Focuses on data + methods together.
Supports inheritance, polymorphism, encapsulation.
Example languages: C++, Java, Python

Example use: Large software systems, real-world modeling

In this framework, all real-world entities are represented by Classes. Objects are instances of classes so each object encapsulates a state and behavior. State implies the fields, attributes of the object and behavior is what you do with the state of the object and they are the methods. Objects interact with each other by passing messages.

Features of object oriented programming:

1. Encapsulation
2. Data abstraction
3. Polymorphism
4. Inheritance
5. Dynamic binding
6. Message passing

1. **Encapsulation** – This is a fundamental feature of Object-Oriented Programming. Here you hide unnecessary details in classes and deliver a simple and clear interface for working. It describes the idea of bundling data and methods that work on that data within one unit. This concept is also often used to hide the internal representation, or state, of an object from the outside

2. **Inheritance** - Inheritance is one of the core concepts of object-oriented programming (OOP) languages. It is a mechanism where you can derive a class from another class for a hierarchy of classes that share a set of attributes and methods. It explains how the class hierarchies develop code readability and support to the reuse of functionality.

3. **Data Abstraction** - to Data abstraction is the reduction of a particular body of data to a simplified representation of the whole. Data abstraction is usually the first step in database design.

4. **Polymorphism** - Polymorphism is an object-oriented programming concept that refers to the ability of a variable, function or object to take on multiple forms. Programming languages that have implemented the OO paradigm are: Ruby, Java, C++, Python, Simula(the first OOP language)

Different ways to achieving polymorphism in C++ program:

- 1) Function overloading
- 2) Operator overloading

```
#include<iostream>
using namespace
std; int main()
{int a=4; a=a<<2;
cout<<"a="<<a<<endl;
return 0;
}
```

5. **Dynamic binding:** Binding refers to linking of procedure call to the code to be executed in response to the call.

Dynamic binding(or late binding) means the code associated with a given procedure call is not known until the time of call at run time.

6. **Message passing:**

An object oriented program consists of a set of objects that communicate with each other. Objects communicate

with each other by sending and receiving information.

A message for an object is a request for execution of a procedure and therefore invoke the function that is called for an object and generates result

Benefits of object oriented programming (OOPs)

- **Data Hiding:** The programmer can hide the data and functions in a class from other classes. It helps the programmer to build the secure programs.
- **Reduced complexity of a problem:** The given problem can be viewed as a collection of different objects. Each object is responsible for a specific task. The problem is solved by interfacing the objects. This technique reduces the complexity of the program design.
- **Easy to Maintain and Upgrade:** OOP makes it easy to maintain and modify existing code as new objects can be created with small differences to existing ones. Software complexity can be easily managed.
- **Modifiability:** it is easy to make minor changes in the data representation or the procedures in an OO program. Changes inside a class do not affect any other part of a program, since the only public interface that the external world has to a class is through the use of methods.

BASIC STRUCTURE OF C++ LANGUAGE :

The program written in C++ language follows this basic structure. The sequence of sections should be as they are in the basic structure. A C program should have one or more sections but the sequence of sections is to be followed.

1. Documentation section
2. Linking section
3. Definition section
4. Global declaration section & class declarations
5. Member function definition
6. Main function section

```
main()
{
Declaration section
Executable section
}
```

1. DOCUMENTATION SECTION : comes first and is used to document the use of logic or reasons in your program. It can be used to write the program's objective, developer and logic details. The documentation is done in C language with /* and */ . Whatever is written between these two are called comments.

2. LINKING SECTION : This section tells the compiler to link the certain occurrences of keywords or functions in your program to the header files specified in this section. e.g. #include<iostream> using namespace std; directive causes the preprocessor to add the contents of the iostream file to the program. It contains declarations for cout and cin.

cout is a predefined object that represents the standard output stream. The operator << is an insertion operator, causes the string in double quotes to be displayed on the screen.

```
cout << "C++";
```

The statement cin>>n; is an input statement and causes the program to wait for the user to type in a number. The number keyed is placed on the variable "n". The identifier cin is a predefined object in C++

that corresponds to the standard input stream. The operator >> is known as extraction operator. It extracts the value from the keyboard and assigns it to the value variable on its right. cin >> 45.5

3. DEFINITION SECTION : It is used to declare some constants and assign them some value.

e.g. #define MAX 25

Here #define is a compiler directive which tells the compiler whenever MAX is found in the program replace it with 25.

4. GLOBAL DECLARATION SECTION : Here the variables and class definitions which are used throughout the program (including main and other functions) are declared so as to make them global(i.e accessible to all parts of program).

A CLASS is a collection of data and functions that act or manipulate the data. The data components of a class are called data members and function components of a class are called member functions

A class also termed as a blue print or prototype that defines the variable or functions common to all objects of certain kind. It is a user defined data type.

e.g.

```
int i; //this declaration is done outside and before main()
```

5. SUB PROGRAM OR FUNCTION SECTION : This has all the sub programs or the functions which our program needs.

```
void display()
{
cout<<"C++ is better than C";
}
```

SAMPLE C++ PROGRAM:

```
#include<iostream>

using namespace std;
void display()
{
cout<<"C++ is better than C";
}
int main()
{
display()
return 0;
}
```

6. MAIN FUNCTION SECTION : It tells the compiler where to start the execution from.

```
main()
{
}
```

main function has two sections

1. declaration section : In this the variables and their data types are declared.
2. Executable section or instruction section : This has the part of program which actually performs the task we need.

namespace:

namespace is used to define a scope that could hold global identifiers.

ex:-namespace scope for c++ standard library.

A classes ,functions and templates are declared within the namespace named std using namespace std;--

>directive can be used.

user defined name space:

syntax for defining name space :

```
namespace namespace_name
```

```
{
//declarations of variables.functions,classes etc...
}
```

ex:

```
#include<iostream>
using namespace std;
namespace sample
{
    int m;
    void display(int n)
    {
        cout<<"in namespace N="<<n<<endl;
    }
}
using namespace sample;
int main()
```

```

{
int a=5;
m=100;
display(200);
cout<<"M in sample name space:"<<sample::m;
return 0;
}

#include<iostream>

```

This directive causes the preprocessor to add content of iostream file to the program. some old versions of C++ used iostream.h .if complier does not support ANSI (american nation standard institute) C++ then use header file iostream.h

Structure in C++

A structure (struct) in C++ is a user-defined data type that groups different types of variables under one name. It is mainly used to represent a record.

Syntax

```

struct structure_name {
    data_type member1;
    data_type member2;
};

```

Example of Structure in C++

```

#include <iostream>
using namespace std;

struct Student {
    int rollNo;
    float marks;
};

int main() {
    Student s1;
    s1.rollNo = 101;
    s1.marks = 85.5;

    cout << "Roll No: " << s1.rollNo << endl;
}

```



```

    cout << "Marks: " << s1.marks << endl;

    return 0;
}

```

Output

Roll No: 101

Marks: 85.5

Features of Structure

- Groups related data together
- Members are public by default
- Can contain data members and member functions
- Supports constructors (in C++)

Comparison: Structure vs Class (C++)

Feature	Structure	Class
Keyword	struct	class
Default access specifier	public	private
Data hiding	Less secure	More secure
Use	Simple data grouping	Complex programs
OOP support	Limited	Full
Inheritance	Supported	Supported

Introduction of Class:

An object oriented programming approach is a collection of objects and each object consists of corresponding data structures and procedures. The program is reusable and more maintainable.

The important aspect in oop is a class which has similar syntax that of structure.

class: It is a collection of data and member functions that manipulate data. The data components of class are called data members and functions that manipulate the data are called member functions.

It can also be called as blue print or prototype that defines the variables and functions common to all objects of certain kind. It is also known as user defined data type or ADT (abstract data type). A class is declared by the keyword `class`.

Syntax:-

```
class class_name
{
Access specifier :
Variable declarations;

Access specifier :
function declarations;

};
```

Access Control:

Access specifier or access modifiers are the labels that specify type of access given to members of a class. These are used for data hiding. These are also called as visibility modes. There are three types of access specifiers:

1. `private`
2. `public`
3. `protected`

1. Private:

If the data members are declared as private access then they cannot be accessed from other functions outside the class. It can only be accessed by the functions declared within the class. It is declared by the key word “private”.

2. public:

If the data members are declared public access then they can be accessed from other functions outside the class. It is declared by the key word “public”.

3. protected:

The access level of protected declaration lies between public and private. This access specifier is used at the time of inheritance.

Note:-

If no access specifier is specified then it is treated by default as private.
The default access specifier of structure is public where as that of a class is “private”

Example:

```
class student
{
private : int roll;
char name[30];
public:
void get_data()
{
cout<<"Enter roll number and name":
cin>>roll>>name;
}
void put_data()
{
cout<<"Roll number:"<<roll<<endl;
cout<<"Name :"<<name<<endl;
}
};
```

Object:-Instance of a class is called object.

Syntax:

```
class_name object_name;
```

Ex:

```
student s;
```

Accessing members:-dot operator is used to access members of class
Object-name.function-name(actual arguments);

Ex:

```
s.get_data();
s.put_data();
```

Note:

- 1.If the access specifier is not specified in the class the default access specifier is private
- 2.All member functions are to be declared as public if not they are not accessible outside the class.

Instance of a class is called as object.

Syntax:

Class_name object name;

Example:

student s;

in the above example s is the object. It is a real time entity that can be used.

Write a program to read data of a student

```
#include<iostream>
```

```
using namespace std;
```

```
class student
```

```
{  
private:  
int roll;  
char name[20];  
public:  
void getdata()
```

```
{cout<<"Enter Roll number:";  
cin>>roll;  
cout<<"Enter Name:";  
cin>>name;  
}  
void putdata()  
{cout<<"Roll no:"<<roll<<endl;  
cout<<"Name:"<<name<<endl;  
}  
};
```

```
int main()  
{  
student s;  
s.getdata();  
s.putdata();  
return 0;  
}
```

Scope Resolution operator:

Scope:- Visibility or availability of a variable in a program is called as scope. There are two types of scope.

- i) Local scope
- ii) Global scope

Local scope: visibility of a variable is local to the function in which it is declared.

Global scope: visibility of a variable to all functions of a program

Scope resolution operator in “::” .

This is used to access global variables if same variables are declared as local and global

```
#include<iostream.h>
int a=5;
void main()
{
int a=1;
cout<<"Local a="<<a<<endl;
cout<<"Global a="<<::a<<endl;
}
```

Class Scope:

Scope resolution operator(::) is used to define a function outside a class.

```
#include <iostream>
using namespace std;
class sample
{
public:
void output(); //function declaration
};
// function definition outside the
class void sample::output() {
cout << "Function defined outside the class.\n";
};

int main() {
sample obj;
obj.output();
return 0;
}
```

Output of program:

Function defined outside the class.

Write a program to find area of rectangle

```
#include<iostream.h>
class rectangle
{
int L,B;
public:
void get_data();
void area();
};
void rectangle::get_data()
{
cout<<"Enter Length of rectangle";
cin>>L;
cout<<"Enter breadth of rectangle";
cin>>B;
}
int rectangle::area()

{
return L*B;
}
int main()
{
rectangle r;
r.get_data();
cout<<"Area of rectangle is"<<r.area();
return 0;
}
```

Array of objects:

Class_name object_name[size];

Where size is the size of array

Ex:

Myclass obj[10];

Write a program to initialize array of objects and print them

```
#include<iostream>
using namespace std;
class MyClass
{
int a;
public:
```

```

void set(int x)
{
a=x;

}
int get()
{
return a;
}
};
int main()
{
MyClass obj[5];
for(int i=0;i<5;i++)
obj[i].set(i);
for(int i=0;i<5;i++)
cout<<"obj["<<i<<"].get():"<<obj[i].get()<<endl;
}

```

Output:

```

obj[0].get():0
obj[1].get():1
obj[2].get():2
obj[3].get():3
obj[4].get():4

```

Objects as Function Arguments: Objects can be used as arguments to

functions This can be done in three ways

- a. Pass-by-value or call by value
- b. Pass-by-address or call by address
- c. Pass-by-reference or call by reference

a. Pass-by-value – A copy of object (actual object) is sent to function and assigned to the object of called function (formal object). Both actual and formal copies of objects are stored at different memory locations. Hence, changes made in formal object are not reflected to actual object.

write a program to swap values of two objects

```

#include<iostream.h>
using namespace std;
class sample2;
class sample1

```

```

{
int a;
public:

void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 x,sample2 y);
};
void sample1::getdata(int x)
{
a=x;
}
class sample2
{
int b;
public:

void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 x,sample2 y);
};
void sample2::getdata(int x)
{
b=x;
}
void display(sample1 x,sample2 y)
{
cout<<"Data in object 1 is"<<endl;
cout<<"a="<<x.a<<endl;
cout<<"Data in object 2 is"<<endl;
cout<<"b="<<y.b<<endl;
}
void swap(sample1 x,sample2 y)
{
int t;
t=x.a;
x.a=y.b;
y.b=t;
}
int main()
{
sample1 obj1;
sample2 obj2;

```



```

obj1.getdata(5);
obj2.getdata(15);
cout<<"Before Swap of data between Two objects\n
"; display(obj1,obj2);
swap(obj1,obj2);
cout<<"after Swap of data between Two objects\n ";
display(obj1,obj2);
}
Before Swap of data between Two objects
Data in object 1 is a=5
Data in object 2 is b=15
after Swap of data between Two objects
Data in object 1 is a=5
Data in object 2 is b=15

```

b. Pass-by-address: Address of the object is sent as argument to function. Here ampersand(&) is used as address operator and arrow (->) is used as de referencing operator. If any change made to formal arguments then there is a change to actual arguments

write a program to swap values of two objects

```

#include<iostream.h>

using namespace std;
class sample2;
class sample1
{
int a;

public:
void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 *x,sample2 *y);
};
void sample1::getdata(int x)
{
a=x;
}
class sample2
{
int b;
public:

```

```

void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 *x,sample2 *y);
};
void sample2::getdata(int x)
{
b=x;
}
void display(sample1 x,sample2 y)
{
cout<<"Data in object 1 is"<<endl;
cout<<"a="<<x.a<<endl;
cout<<"Data in object 2 is"<<endl;

cout<<"b="<<y.b<<endl;
}
void swap(sample1 *x,sample2 *y)
{
int t;
t=x->a;
x->a=y->b;
y->b=t;
}
int main()
{
sample1 obj1;
sample2 obj2;
obj1.getdata(5);
obj2.getdata(15);
cout<<"Before Swap of data between Two objects\n ";
display(obj1,obj2);
swap(&obj1,&obj2);
cout<<"after Swap of data between Two objects\n ";
display(obj1,obj2);
}

```

Before Swap of data between Two objects

Data in object 1 is a=5

Data in object 2 is b=15

after Swap of data between Two objects

Data in object 1 is a=15

Data in object 2 is b=5

c.Pass-by-reference: A reference of object is sent as argument to function. Reference to a variable provides alternate name for previously defined variable. If any change made to reference variable then there is a change to original variable. A reference variable can be declared as follows:

Datatype & reference variable =variable;

Ex:

```
int x=5;
```

```
int &y=x;
```

Write a program to find sum of n natural numbers using reference variable

```
#include<iostream.h>
using namespace std;
int main()
{
int i=0;
int &j=i;
int s=0;
int n;
cout<<"Enter n:";
cin>>n;
while(j<=n)
{
s=s+i;

i++;
}
cout<<"sum="<<s<<endl;
}
```

Output:

Enter n:10

sum=55

write a program to swap values of two objects

```
#include<iostream.h>
using namespace std;
class sample2;
class sample1
{
int a;
```

```
public:
void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 &x,sample2 &y);
};
```

```
void sample1::getdata(int x)
{
a=x;
}
class sample2
```

```
{
int b;
public:
void getdata(int x);
friend void display(sample1 x,sample2 y);
friend void swap(sample1 &x,sample2 &y);
};
```

```
void sample2::getdata(int x)
{
b=x;
}
```

```
void display(sample1 x,sample2 y)
{
cout<<"Data in object 1 is"<<endl;
cout<<"a="<<x.a<<endl;
cout<<"Data in object 2 is"<<endl;
cout<<"b="<<y.b<<endl;
}
```

```
void swap(sample1 &x,sample2 &y)
{
int t;
t=x.a;
x.a=y.b;
y.b=t;
}
```

```
int main()
```

```
{
sample1 obj1;
sample2 obj2;
obj1.getdata(5);
```

```
obj2.getdata(15);  
cout<<"Before Swap of data between Two objects\n ";  
display(obj1,obj2);  
swap(obj1,obj2);  
cout<<"after Swap of data between Two objects\n ";  
display(obj1,obj2);  
}
```

Output:

Before Swap of data between Two objects

Data in object 1 is a=5

Data in object 2 is b=15

after Swap of data between Two objects

Data in object 1 is a=15

Data in object 2 is b=5